

Concepts Reference

For the AlgoChowk Quant Engineer NIFTY Recovery Assignment

Every term below is explained in the order it actually shows up in your pipeline: data → event detection → backtest mechanics → descriptive stats → inferential stats → robustness → out-of-sample → final backtest. Each entry uses the real numbers from your terminal output so you can map the definition straight onto what you already saw.

1. Data & Returns

OHLC

Open, High, Low, Close — the four prices recorded for each trading day. Open/Close are the first and last traded prices of the session; High/Low are the max and min during the session.

In your run: Your CSV has one row per trading day: Date, Open, High, Low, Close (plus Volume).

Daily return

The percentage change in Close price from one day to the next: $return = Close_t / Close_{t-1} - 1$. We work in returns, not raw prices, because a Rs.100 move means something very different when NIFTY is at 3,000 versus at 25,000 — returns put every era on the same scale.

In your run: `df['ret'] = df['Close'].pct_change()` in `event_engine.py`.

Row count / date range / trading-day coverage

Basic sanity checks before trusting any analysis: how many rows you actually have, what dates they span, and whether the count-per-year looks like a real trading calendar (NSE runs roughly 245–250 sessions/year).

In your run: 4,663 rows, 2007-09-17 to 2026-09-18, ~245/year — passed.

Data validation checks

Before any modeling: duplicate dates (same day appearing twice), non-monotonic ordering (dates out of sequence), missing values, and OHLC inconsistencies (e.g. High lower than Open/Close, which is physically impossible for a valid session).

In your run: Your run: 0 duplicates, monotonic increasing, 0 missing, 0 OHLC inconsistencies.

Suspicious observations

Days flagged for manual review because the move is unusually large — not necessarily wrong, just worth checking they're real events rather than data errors before you build on top of them.

In your run: 3 days flagged at $|return| > 10\%$: 2008-10-24 (financial crisis), 2009-05-18 (election-result rally), 2020-03-23 (COVID crash) — all real, kept in the data.

2. Event Detection

Event

A day that meets your definition of “significant fall.” The assignment doesn't define this for you — you pick a rule and justify it.

Threshold

The cutoff return that defines an event. A day is an event if its return is at or below the threshold.

In your run: `threshold = -3%` → any day with `return ≤ -3%` counts as an event. Lowering it to `-2%` catches more (smaller) falls; raising it to `-4%` catches fewer, more extreme ones.

n (sample size)

The number of observations going into a calculation. More n generally means more trustworthy statistics; very small n means wide uncertainty even if the average looks impressive.

In your run: $n=79$ events at `-3%` ($n=59$ after removing clustered ones) — small enough that none of your confidence intervals or p-values came out sharp.

Independent event / clustering / min_gap_days

Crashes don't arrive as isolated, unrelated days — a -8% day is often followed by more $\geq 3\%$ down days as the same crisis continues (e.g. Jan 2008). If you count each of those as a separate, independent data point, you're really just counting the same crash multiple times, which inflates your apparent sample size and can bias results toward whatever that one crisis did. `min_gap_days` is the minimum spacing (in trading days) required for two events to be treated as independent.

In your run: 2008-01-21, 22, 24 are 3 events 1–2 days apart → `flagged_is_independent=False`; treated as one cluster, not three separate falls.

3. Backtest Mechanics (Entry, Exit, Holding Period)

Look-ahead bias

Using information you couldn't actually have had at the time of the decision. The classic mistake here: buying at the same day's Close when that Close is exactly what confirmed the event — by the time you know the day was a -3% day (at market close), you can no longer trade at that day's price.

Entry price / entry lag

The price you'd realistically transact at. `entry_lag=1` means: event confirmed at Close of day t , so you enter at the Open of day $t+1$ — the next price you could actually act on.

In your run: `entry_price = df.loc[event_idx + 1, 'Open']` in `compute_forward_returns()`.

Holding period (N)

How many trading days you stay in the position before exiting, measured from entry. Different holding periods test different versions of the hypothesis ("recovers fast" vs. "recovers eventually").

In your run: Tested $N = 1, 3, 5, 10$ trading days.

Forward return

The return earned from entry to exit for a given holding period: $\text{Close at exit} \div \text{entry price} - 1$.

In your run: 5-day forward return after a -3% event averaged +0.42% (all events) in your first run.

Exit

The price at which the position is closed — here, the Close price N days after entry.

4. Baseline

Baseline

The same forward-return calculation applied to every day in the dataset, not just event days. This is the control group: NIFTY drifts upward over most multi-year windows regardless of any crash, so a positive average return after an "event" means nothing on its own — it only means something if it beats what an ordinary, randomly chosen day would have returned over the same horizon.

In your run: 10-day baseline mean was +0.33% — almost identical to the 10-day event mean (+0.30%). The event pattern you're hunting for has to clear this bar, not zero.

5. Descriptive Statistics

Mean vs. median

Mean is the arithmetic average; median is the middle value when sorted. Returns are often skewed by a few extreme days, so when mean and median disagree a lot, it's a sign a handful of outliers are driving the mean.

Win rate

The fraction of events where the forward return was positive — a simple "did it go up more often than not" measure, distinct from how *much* it went up.

In your run: 56.96% at 5 days (all events) — a bit above a coin flip, but see the confidence interval before reading much into that.

Practical recovery rate

Stricter than win rate: the fraction of events where price actually climbed back to or above its pre-fall level (the Close right before the event) within N days — i.e. the loss was fully erased, not just "the next few days were slightly green."

In your run: Only 30.38% of events had fully recovered within 5 days even though the win rate over the same days was 57% — "positive average return" and "actually back to even" are different claims.

Standard deviation / volatility

A measure of how spread out returns are around the mean. High volatility means individual outcomes vary widely even if the average looks stable — directly relevant to how wide your confidence intervals end up being.

6. Inferential Statistics (Is the Pattern Real?)

Confidence interval (CI)

A range that plausibly contains the true average effect, given the sample you have. A 95% CI roughly means: if you repeated this sampling process many times, about 95% of the intervals you'd construct would contain the true value. A CI that spans both negative and positive numbers is telling you "we can't even confirm the sign of the effect" — not just its size.

In your run: 5-day event mean +0.42% with 95% CI [-1.01%, +1.84%] — the interval includes zero and even negative values, so you can't confidently say the true effect is positive at all.

Bootstrap

A way to build a confidence interval without assuming returns follow any particular distribution (like a bell curve). You repeatedly resample your own data *with replacement* (drawing the same size sample over and over, sometimes picking the same day twice), recompute the mean each time, and use the spread of those thousands of resampled means as your CI. Useful here because return data is skewed and fat-tailed, not normal.

In your run: `bootstrap_mean_ci()` in `stats_engine.py` runs 10,000 resamples per number reported.

Permutation test

A way to test "is the difference between event-days and baseline-days real, or could it just be random labeling?" You pool both groups together, then repeatedly shuffle which observations get called "event" vs. "baseline" and recompute the difference in means each time. The p-value is the fraction of those random shufflings that produced a gap at least as large as what you actually observed. If shuffled labels regularly produce gaps just as big, your real gap isn't special.

In your run: 5-day permutation $p=0.37$ (all events) — roughly 37% of random relabelings produced as big a gap as the real one, so the real gap isn't unusual.

p-value

The probability of seeing a result at least this extreme *if there were actually no real effect* (the "null hypothesis"). Small p-value (conventionally <0.05) = the result would be unlikely under pure chance, which is evidence (not proof) against the null. It is **not** the probability that the hypothesis is true.

In your run: All your p-values across holding periods were 0.12–0.97 — none clear the conventional 0.05 bar.

Welch's t-test

A standard test comparing the means of two groups (here: event-day returns vs. baseline returns) that doesn't assume the two groups have equal variance. Reported alongside the permutation test as a common reference point, with the caveat that it does assume roughly normal data — a real stretch for skewed, overlapping return series — which is why the permutation test is treated as the primary evidence.

Statistical significance vs. economic significance

A result can be statistically detectable (unlikely to be pure chance) but still too small to matter for actual trading once you account for costs — or vice versa: a juicy-looking average return with so few observations that it's statistically indistinguishable from noise. The assignment explicitly warns about this; your results sit in the second case — not statistically significant, so the economic-significance question doesn't even arise yet.

7. Robustness, Data Snooping & Overfitting

Data snooping / multiple testing

If you test enough different thresholds, holding periods, or parameter combinations, some will look "significant" purely by chance — the same way flipping enough coins produces a run of 5 heads in a row eventually. Testing many combinations without accounting for this inflates your false-positive risk.

Bonferroni correction

One standard fix for multiple testing: divide your significance threshold (usually 0.05) by the number of tests you ran, and only trust results below that stricter bar.

In your run: Sweeping 6 thresholds $\times$ 4 holding periods = 24 tests $\rightarrow$ Bonferroni alpha = $0.05/24 \approx 0.0021$ instead of 0.05 — a much higher bar to clear.

Post-hoc parameter selection / overfitting

Picking your threshold or holding period *after* seeing which one produced the best-looking result. This all but guarantees an impressive backtest that won't replicate going forward, because you've fit the parameter to noise in this specific historical sample rather than to a real, stable effect. The fix is to report the full sweep honestly (including the mediocre cells) rather than presenting only your best result.

8. Out-of-Sample (OOS) Validation

Research / Development split vs. Out-of-Sample split

You split your data chronologically into two blocks: an earlier "research" period where you're allowed to look at the data, choose your threshold, and tune parameters — and a later "out-of-sample" period you deliberately don't look at until the very end, then run your already-finalized method on once, unchanged. If the pattern only exists in the research window and disappears in the OOS window, that's strong evidence it was overfitting rather than a real, persistent effect.

In your run: config.py currently sets research_end_date = 2019-12-31 — research period 2007–2019, out-of-sample 2020–2026 (which conveniently includes the COVID crash as a genuine unseen stress test).

9. Event-Driven Backtest (If Results Support It)

Transaction costs / slippage

Real-world frictions that eat into returns: brokerage/exchange fees (transaction costs) and the gap between the price you expected and the price you actually got when the order filled (slippage). A strategy that only looks good before costs is not a strategy.

In your run: config.py currently assumes 5 bps (0.05%) per side as a simple placeholder.

Number of trades

How many times the strategy actually entered and exited — directly tied to n above; with only 59–79 events over 19 years, this is a low-frequency strategy, which matters for how much total cost and how much statistical power you have.

Cumulative performance

The compounded return of the strategy across all trades over the full test period, usually plotted as a growth curve.

Maximum drawdown

The largest peak-to-trough decline in the strategy's cumulative value over the test period — a measure of the worst pain an investor following the strategy would have lived through, separate from its average return.

Quick Reference Table

Term	One-line meaning	Where in your code
Threshold	Cutoff return defining an event	config.py: event_threshold
Event	Day meeting the threshold	detect_events()
Independent event	Not within min_gap of a prior event	is_independent column
Entry lag	Days after event before you can enter	config.py: entry_lag
Holding period (N)	Days held before exit	config.py: holding_periods
Forward return	Return from entry to exit	fwd_ret_{N}d column
Baseline	Same calc on ALL days, not just events	compute_baseline_forward_returns()
Win rate	% of events with positive forward return	(event_ret > 0).mean()
Recovery rate	% back to pre-fall price within N days	recovered_{N}d column
Bootstrap CI	Resampling-based confidence interval	bootstrap_mean_ci()
Permutation p-value	Chance the gap is due to random labeling	permutation_test_diff_means()
Bonferroni alpha	Stricter significance bar for many tests	robustness_sweep.py
Out-of-sample	Held-out later period, tested once	config.py: research_end_date

Bottom line for where you are right now: none of your permutation p-values have cleared 0.05, and the gap between event-day and baseline returns is well within what random noise produces at n=59–79. That's a legitimate, reportable finding — the assignment explicitly says a negative result is acceptable. The next steps (threshold sweep, OOS split) exist to check whether that null result is stable, not to go hunting for a parameter combination that flips it to "significant."